

HackLISP-0 Programmer's Manual

Grammar

program: object*

object: list | atom

list: `( ` (object | `.`)* `)`

atom: symbol | integer

symbol: symbol-start symbol-middle*

symbol-start: All characters other than `( `)` , integers, `.` , and whitespace

symbol-middle: All characters other than `( `)` and whitespace

integer: (ASCII 48~57)

Tokenization notes

Dots must be separated by whitespace, otherwise they're considered part of the symbol.

Integers must have a whitespace or a paren after them; you cannot have a symbol or dot immediately following.

Data types

Cons cell

A cons cell is a pair, represented in text as an **s-expression**

(*head* . *tail*).

Both *head* and *tail* are stored as pointers, to objects of any data type.

A cons cell evaluates to the result of calling its head with its tail as the arguments.

List

A list is a list of things. It is represented as a cons cell, whose *tail* is another cons cell. The last element of the list is represented by (*head* . NIL). For example:

(A B C D E) = (A . (B . (C . (D . (E . NIL)))))

A list evaluates to the result of calling its head with its tail as the arguments.

Atom

An *atom* is a basic thing, it's not made out of any other Lisp objects.

Symbol

A symbol is stored as a pointer to a null-terminated C string on the atom stack. It can represent whatever you want. The following symbols are predefined; you cannot redefine them:

`NIL TRUE quote cond = cons atom? head tail lambda read print integer?`
`+ - * / & | mem-get mem-put!`

A symbol evaluates to whatever value is bound to it. The program will crash if you evaluate a symbol that's not bound to it.

As an exception, **NIL** and **TRUE** evaluate to themselves. `()` and **NIL** are indistinguishable.

Integer

An integer represents, well, an integer. It evaluates to itself.

Basic builtins

These basically do the “list programming” part of LISP.

quote

`(quote x)` returns x , without evaluating it.

```
> (quote a)
a
> (quote (a b c))
(a b c)
```

Important: `quote` is the only builtin function which does not evaluate its arguments.

cond

`(cond (p1e1)...(pnen))`

The p_k are evaluated in order until one returns **TRUE**. When that happens, the value of e_k is returned as the value of the whole expression.

```
> (cond (NIL (quote no)) (TRUE (quote yes))))
yes
> (cond (TRUE (quote first)) (TRUE (quote second))))
first
```

`=`

`(= x y)` returns **TRUE** if x and y are both atoms and are the same, or if they are lists stored in the same memory position. Otherwise, it returns **NIL**.

```
> (= (quote a) (quote a))
TRUE
> (= (quote b) (quote a))
NIL
```

cons

`(cons x y)` returns $(x . y)$.

```
> (cons (quote a) (quote (b c)))
(a b c)
> (cons (quote a) (quote b))
(a . b)
```

atom?

(**atom?** x) returns **TRUE** if x is an atom, otherwise it returns **NIL**.

```
> (atom? (quote (a b)))  
NIL  
> (atom? 3)  
TRUE  
> (atom? (quote atom?))  
TRUE
```

head

(**head** x) returns the head of x (its first element).

```
> (head (quote (a b)))  
a
```

Most Lisps call it **car** (Contents of the Address Register).

tail

(**tail** x) returns the tail of x :

- If it's a cons cell, its second element
- If it's a list, everything after the first element

```
> (tail (quote (a b c)))  
(b c)  
> (tail (quote (a . b)))  
b
```

Most Lisps call it **cdr** (Contents of the Decrement Register).

Extra builtins

read

(**read**) waits for the user to enter in an s -expression, then it parses the s -expression and returns it.

print

(**print** x) prints the value x (as an s -expression) to the screen.

(**print**) with no arguments makes a new line.

It returns the arguments.

integer?

(**integer?** x) returns **TRUE** if x is an integer and **NIL** if not.

+

(+ x y) returns $x + y$.

—

(— x) returns $-x$.

$(- x y)$ returns $x - y$.

$*$

$(* x y)$ returns $x * y$.

$/$

$(/ x y)$ returns $\frac{x}{y}$.

$\&$

$(\& x y)$ returns the bitwise AND of x and y .

$|$

$(| x y)$ returns the bitwise OR of x and y .

mem-get

(mem-get x) returns the value at $\text{RAM}[x]$.

mem-put!

(mem-put! $x y$) sets $\text{RAM}[x]$ to y .

Lambdas

A lambda is expressed as

$$(\text{lambda } (p_1 \dots p_n) e),$$

where $p_1 \dots p_n$ are the *parameters* and e is an expression. A *function call* is an expression

$$((\text{lambda } (p_1 \dots p_n) e) a_1 \dots a_n).$$

Note that the keyword **lambda** is not actually a keyword; you can replace it by anything of your choosing and it will be ignored.

1. Each a_i is evaluated.
2. A new environment, consisting of the outside environment plus the following bindings is created:

$$\begin{array}{l} p_1 \rightarrow a_1 \\ p_2 \rightarrow a_2 \\ \vdots \\ p_n \rightarrow a_n \end{array}$$

3. e is evaluated in said environment.
4. The expression returns the retval of e .

If an expression is of the form

$$(f a_1 \dots a_n).$$

where f is bound to a lambda, then f is replaced by said lambda and evaluated.

The default environment, i.e. at the top level, is

NIL $\rightarrow$ NIL
TRUE $\rightarrow$ TRUE.

Key idea: lambdas by themselves are Turing-complete! (see lambda calculus)

But LISP's lists make it more convenient.

Useful tricks

Recursion and naming

```
((lambda (a b) (a 1))
 ; Note that they don't actually have to start with lambda-[name]
 ; Since we don't actually check for the lambda keyword we can do this
 (quote (lambda-a (x) (cons x (b x))
 (quote (lambda-b (x) (cons x (a x))
 )
```

Imperative programming

```
((lambda (main prog) (main))
 (quote (lambda-main () (prog
 (print (quote (Hello world)))
 (print)
 (print (quote (Goodbye)))
 (print)
 )))
 ; It pretends to take no arguments,
 ; but it actually takes infinitely many
 (quote (lambda-prog () NIL))
 )
```